What Is Recursion?

Imagine you are standing in a long line at a movie theater and want to know what position you are in, without leaving your spot. You tap the person in front of you and ask: "What position are you in?" That person doesn't know either, so they tap the person in front of them and ask the exact same question. This repeats all the way to the very first person at the ticket counter.

The person at the front says: "I am #1!" Now the answers travel backward: the second person adds 1 and says "I am #2!", the third says "I am #3!", until the person in front of you turns around and says "I am #14," allowing you to know you are #15.

That is recursion. In programming, a recursive function is simply a function that solves a big problem by asking a copy of itself to solve a slightly smaller version of the exact same problem.

> **The Two Golden Rules of Every Recursive Function**
>
> 1. **The Base Case (The Stopping Point**): The simplest possible version of the problem that the function can answer immediately without calling itself again (like the person at the very front of the line saying 'I am #1'). Without a base case, a recursive function runs forever until the computer runs out of memory.
> 2. **The Recursive Step (Shrinking the Problem**): Calling the same function with a smaller or simpler input so that it gets closer and closer to the Base Case on every single step.

Part 2: What Happens Inside the Computer

Many beginners wonder: "If a function calls itself, doesn't changing a variable like n overwrite the old value of n?" The answer is NO-because of how computer memory works.

Every single time a function is called-even if it is the exact same function calling itself-the computer builds a brand-new, sealed workspace in memory called a Frame Object (or Stack Frame). Think of a Frame Object as a fresh desk with its own private sticky notes.

What Is Inside a Frame Object?

In languages like Python, a Frame Object is an actual object stored in memory. Each frame holds four essential items:

- **1. f_locals (Private Local Variables):** A private dictionary holding the parameters and variables belonging ONLY to that specific call. Frame 2 can change its own variable n without ever touching Frame 1's variable n.
- **2. f_lasti (Instruction Pointer / Bookmark):** When Frame 1 calls Frame 2, Frame 1 freezes right in the middle of its code. f_lasti is a bookmark remembering the exact line number where Frame 1 paused so it knows where to continue when Frame 2 finishes.
- **3. f_back (Return Address / Caller Link):** An arrow pointing to the frame right underneath it. When the top frame finishes and executes return, f_back tells the computer which frame should receive the answer.
- **4. f_code & Evaluation Scratchpad:** The instructions the function is running, plus a temporary scratchpad for doing math (like holding the left half of an addition while waiting for the right half).

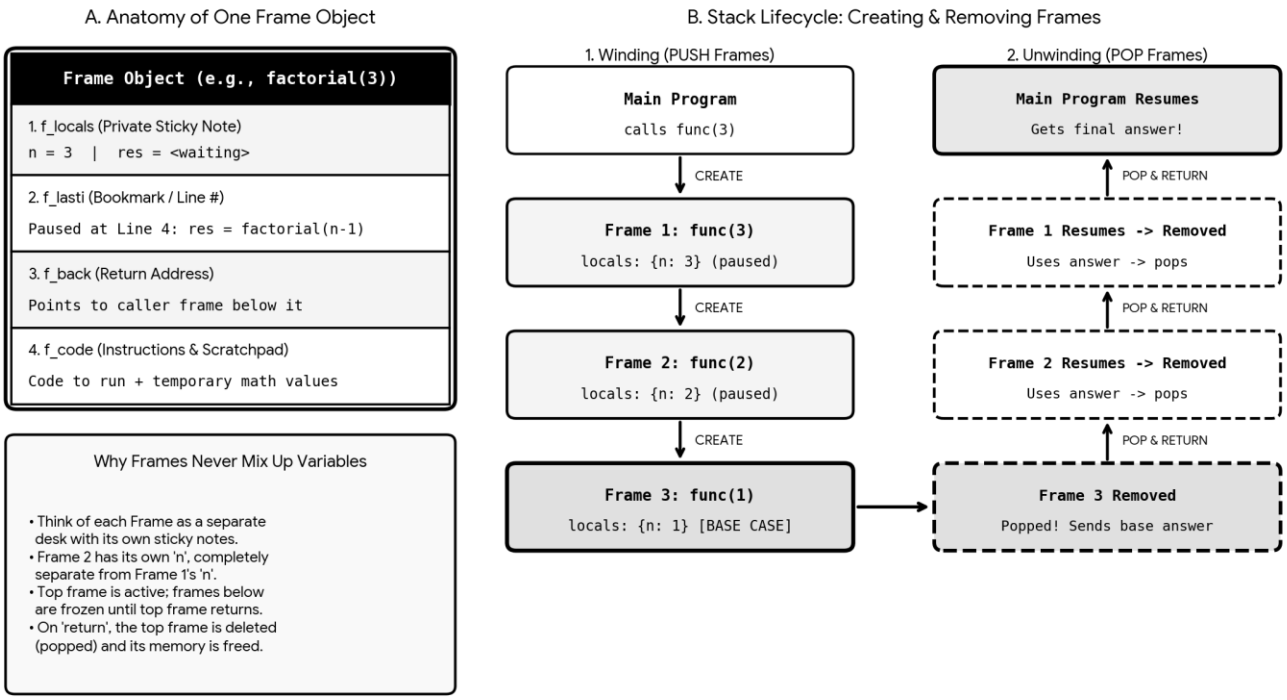
How Frames Are Created (PUSH) and Removed (POP)

The computer organizes all active Frame Objects in a vertical pile called the Call Stack. It works just like a books stacked one on top of the other :

- **Creating a Frame(also called PUSH):** Whenever a function is called, a new Frame Object is created and placed on the very top of the stack of frames. Only the frame at the very top of the stack is actively running; every frame beneath it is frozen and waiting.
- **Removing a Frame(Also called POP):** The moment the top frame hits a return statement (or reaches its final line), its job is completely done. The computer sends its return value down to the frame below it, and the finished frame is immediately destroyed from the stack, freeing its memory.

Figure 1: Inside a Frame Object & How the Call Stack Works

Every function call creates an independent Frame Object in memory (Last-In, First-Out Stack)



Step-by-Step Examples With Frame Traces

Example 1: Countdown (The Simplest Possible Recursion)

Let's start with a function that prints numbers from n down to 1 and then prints 'Blastoff!'. Notice that this function does not even do any math after the recursive call returns-making it the cleanest way to watch frames being created and destroyed.

Example 1 - Python Code: countdown(3)

```
def countdown(n):
    if n == 0:              # Line 2: Base Case (stopping point)
        print("Blastoff!")  # Line 3
        return              # Line 4: Destroys this frame and returns None
    print(n)                # Line 5
    countdown(n - 1)        # Line 6: Creates a new frame for (n - 1)

countdown(3)                # Line 8: Starts the chain
```

Watch what happens to the Frame Objects and their local variables step-by-step:

- **Frame 1 - countdown(3):** Created with f_locals = {'n': 3}. Since 3 != 0, it prints 3, then calls countdown(2) at Line 6. Frame 1 freezes at Line 6.
- **Frame 2 - countdown(2):** Created on top of Frame 1 with f_locals = {'n': 2}. Prints 2, calls countdown(1) at Line 6, and freezes.
- **Frame 3 - countdown(1):** Created on top of Frame 2 with f_locals = {'n': 1}. Prints 1, calls countdown(0) at Line 6, and freezes.
- **Frame 4 - countdown(0) [BASE CASE]:** Created with f_locals = {'n': 0}. Now n == 0 is True! It prints 'Blastoff!' and hits return. Frame 4 is immediately popped and destroyed.
- **Unwinding (Frames 3, 2, 1 Popped):** Frame 3 wakes up at Line 6, sees no more lines of code left in the function, and automatically pops. Frame 2 does the same, then Frame 1 pops. All 4 frames are now completely gone from memory.

Figure 2: Example 1 — countdown(3) Frame Objects & Locals Over Time

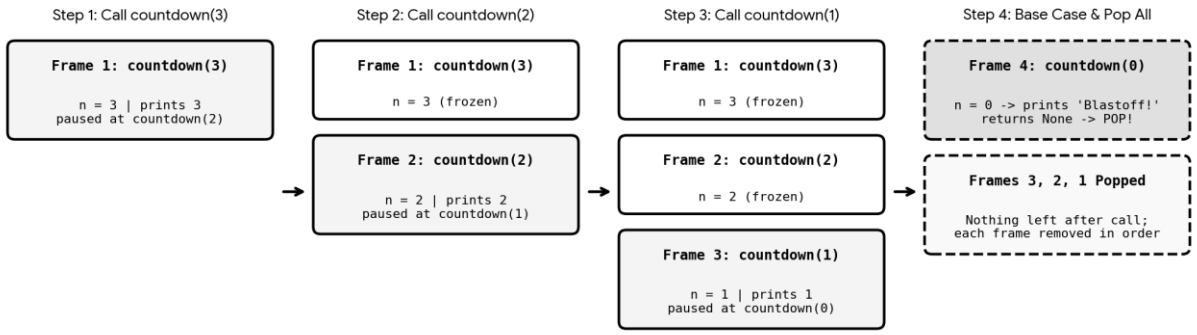


Figure 2: Step-by-step creation and removal of Frame Objects during countdown(3).

Example 2: Factorial

The factorial of a positive integer n (written n!) is the product of all numbers from n down to 1. For example, 4! = 4 × 3 × 2 × 1 = 24. Notice how 4! is really just 4 × 3!, and 3! is 3 × 2!, down to the Base Case 1! = 1.

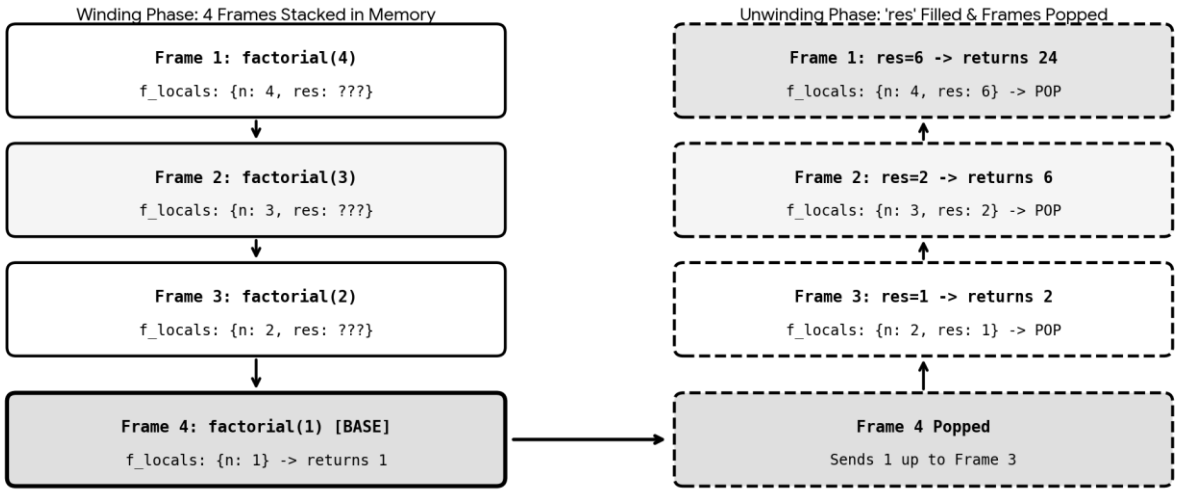
Example 2 - Python Code: factorial(4)

```
def factorial(n):
    if n == 1:              # Line 2: Base Case
        return 1            # Line 3
    res = factorial(n - 1)  # Line 4: Pauses here while child frame computes (n-1)!
    return n * res          # Line 5: Resumes here once 'res' receives a value

ans = factorial(4)          # Line 7: Final ans becomes 24
```

Unlike countdown, each frame here has a second local variable named res that remains unassigned until the frame above it returns an answer:

Figure 3: Example 2 — factorial(4) Frame Creation, Local Updates & Removal



Example 3: Greatest Common Divisor - GCD (Two Changing Locals)

The Greatest Common Divisor (GCD) of two numbers a and b is the largest whole number that divides both evenly (for example, gcd(48, 18) = 6). Euclid discovered over 2,000 years ago that gcd(a, b) is always equal to gcd(b, a % b), where a % b is the remainder when dividing a by b. Once the remainder b reaches 0, a is the answer!

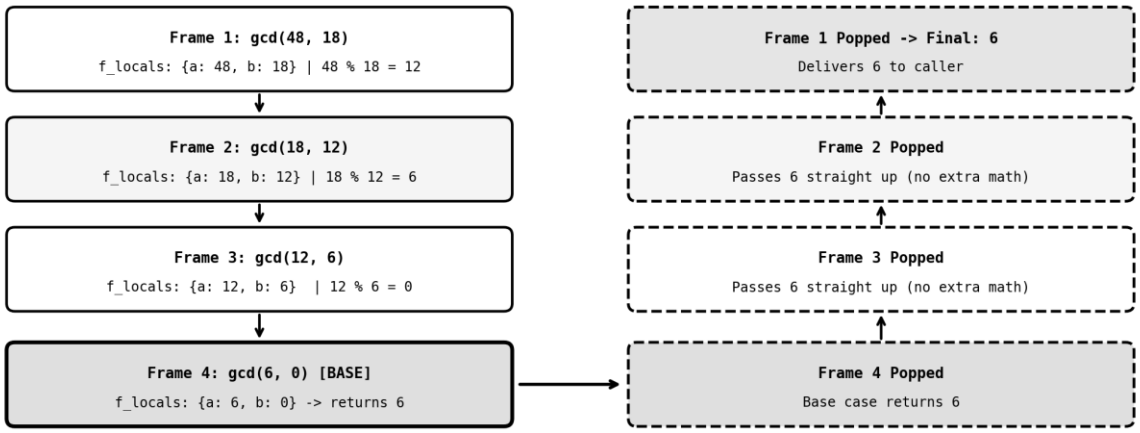
Example 3 - Python Code: gcd(48, 18)

```
def gcd(a, b):
    if b == 0:              # Line 2: Base Case (no remainder left!)
        return a            # Line 3
    return gcd(b, a % b)    # Line 4: New frame gets (old b, remainder)
```

Here, every new Frame Object has TWO local variables (a and b) that shift and shrink on every call. Even better: because Line 4 directly returns the result of the next call without extra multiplication afterward, the Base Case answer (6) travels straight down through the popped frames unchanged:

Depth	Frame Object	f_locals Inside Frame	Remainder (a % b)	Action / Return Value
1	`Frame 1: gcd(48, 18)`	`{a: 48, b: 18}`	48 % 18 = 12	**Creates Frame 2; waits for answer -> gets 6, POPPED**
2	`Frame 2: gcd(18, 12)`	`{a: 18, b: 12}`	18 % 12 = 6	**Creates Frame 3; waits for answer -> gets 6, POPPED**
3	`Frame 3: gcd(12, 6)`	`{a: 12, b: 6}`	12 % 6 = 0	**Creates Frame 4; waits for answer -> gets 6, POPPED**
4	`Frame 4: gcd(6, 0)`	`{a: 6, b: 0}`	N/A (b == 0)	**BASE CASE! Returns a = 6 and immediately POPPED**

Figure 4: Example 3 — gcd(48, 18) Passing Directly Back Up the Stack



Example 4: Fibonacci (Branching Recursion & Frame Reuse)

In the Fibonacci sequence (0, 1, 1, 2, 3, 5, 8...), each number is the sum of the two numbers before it: fib(n) = fib(n - 1) + fib(n - 2), with two Base Cases: fib(0) = 0 and fib(1) = 1.

This example reveals a crucial insight about computer memory: when a function makes TWO recursive calls, it does NOT run them at the same time. It runs the first call completely (creating and popping all its frames) BEFORE it ever starts the second call!

Example 4 - Python Code: fib(4)

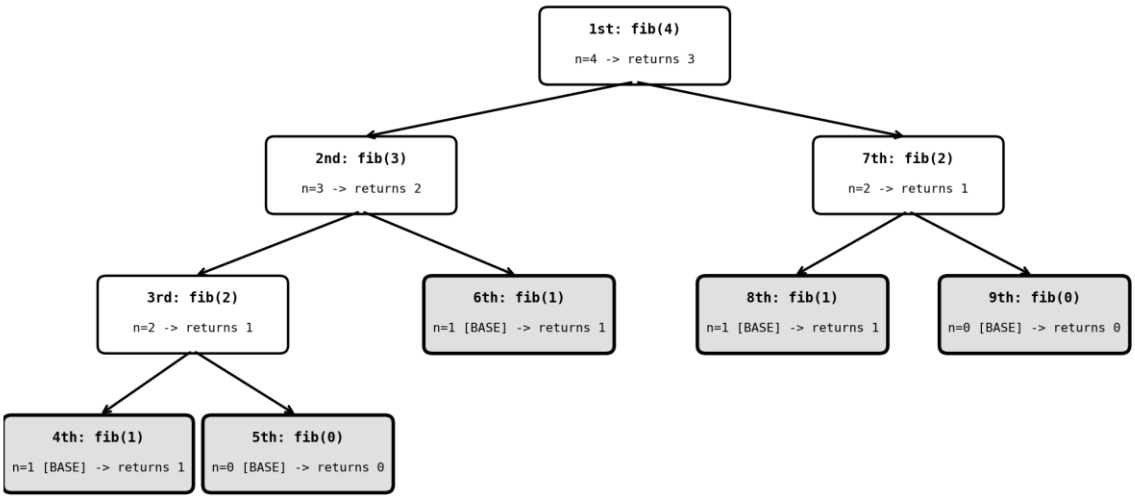
```
def fib(n):
    if n <= 1:              # Line 2: Two Base Cases: fib(0)=0, fib(1)=1
        return n            # Line 3
    left = fib(n - 1)       # Line 4: Runs FIRST (creates & pops entire left branch)
    right = fib(n - 2)      # Line 5: Runs SECOND (reuses freed stack space!)
    return left + right     # Line 6: Combines both answers
```

Look at what happens inside Frame 1 (fib(4)):

- **Step A (Left Branch Runs First):** Frame 1 pauses at Line 4 and calls fib(3). That creates fib(2), which calls fib(1) (returns 1, pops) and fib(0) (returns 0, pops). Then fib(2) returns 1 and pops. Next, fib(3) calls fib(1) (returns 1, pops), and finally fib(3) returns 2 and pops. Frame 1 now sets left = 2.
- **Step B (Right Branch Reuses Memory):** Now Frame 1 moves to Line 5 and calls fib(2). All the frames from the left branch have ALREADY been destroyed, so the computer reuses that exact same stack memory for fib(2), fib(1), and fib(0). Once fib(2) returns 1 and pops, Frame 1 sets right = 1.
- **Step C (Final Sum & Removal):** Frame 1 now has f_locals = {'n': 4, 'left': 2, 'right': 1}. At Line 6 it returns 2 + 1 = 3, and Frame 1 is popped.

Figure 5: Example 4 — fib(4) Branching Calls (Frames Are Reused After Popping!)

Total frames created = 9, but at most 4 frames exist on the stack at the same time.



Notice: Left branch (2nd–6th calls) finishes and pops completely BEFORE the right branch (7th–9th calls) is created!